ASSIGMENT

PROGRAMMING USING C

WEEK – 15

NAME : K.Venkateshwari

ROLL NO : 2116240801373

DEPT & SEC :ECE F

Week – 15

Given an array of integers, reverse the given array in place using an index and loop rather than a built-in function.

Example

arr = [1, 3, 2, 4, 5]

Return the array *[5, 4, 2, 3, 1]* which is the reverse of the input array.

```
 * The function accepts INTEGER_ARRAY arr as parameter.
 */

/*
 * To return the integer array from the function, you should:
 *      - Store the size of the array to be returned in the result_count variable
 *      - Allocate the array statically or dynamically
 *
 * For example,
 * int* return_integer_array_using_static_allocation(int* result_count) {
 *      *result_count = 5;
 *
 *      static int a[5] = {1, 2, 3, 4, 5};
 *
 *      return a;
 * }
 *
 * int* return_integer_array_using_dynamic_allocation(int* result_count) {
 *      *result_count = 5;
 *
 *      int *a = malloc(5 * sizeof(int));
 *
 *      for (int i = 0; i < 5; i++) {
 *          *(a + i) = i + 1;
 *      }
 *
 *      return a;
 * }
 *
 */
int* reverseArray(int arr_count, int *arr, int *result_count) {
    int *b = (int *)malloc(arr_count * sizeof(int));
    if(b == NULL){
        return NULL;
    }
    *result_count = arr_count;
    for(int i =0;i<arr_count;i++){
        b[i]=arr[arr_count -1 -i];
    }
    return b;
```

	Test	Expected	Got	

✓	int arr[] = {1, 3, 2, 4, 5};
	int result_count;
	int* result = reverseArray(5, arr, &result_count);
	for (int i = 0; i < result_count; i++)
	printf("%d\n", *(result + i));
		5
		4
		2
		3
		1
			5
			4
			2
			3
			1
				✓

Passed all tests! ✓

An automated cutting machine is used to cut rods into segments. The cutting machine can only hold a rod of *minLength* or more, and it can only make one cut at a time. Given the array *lengths[]* representing the desired lengths of each segment, determine if it is possible to make the necessary cuts using this machine. The rod is marked into lengths already, in the order given.

```
 * The function accepts following parameters:
 *  1. LONG_INTEGER_ARRAY lengths
 *  2. LONG_INTEGER minLength
 */

/*
 * To return the string from the function, you should either do static allocation or dynamic allocation
 *
 * For example,
 * char* return_string_using_static_allocation() {
 *     static char s[] = "static allocation of string";
 *
 *     return s;
 * }
 *
 * char* return_string_using_dynamic_allocation() {
 *     char* s = malloc(100 * sizeof(char));
 *
 *     s = "dynamic allocation of string";
 *
 *     return s;
 * }
 *
 */
char* cutThemAll(int lengths_count, long *lengths, long minLength) {
    long totallength=0;
    for (int i=0;i< lengths_count;i++){
        totallength += lenghts[i];
    }
    if(totallength<minLength){
        return "Impossible";
    }
    long remainingLength=totallength;
    for(int i=0;i<lenghts_count;i++){
        remainingLength-=lenghts[i];
        if(remainingLength>=minLength){
            return "Possible";
        }
    }
    return "Impossible";
}
```

Syntax Error(s)

```
__tester__.c: In function ‘cutThemAll’:
__tester__.c:36:24: error: ‘lenghts’ undeclared (first use in this function); did you mean ‘lengths’?
   36 |         totallength += lenghts[i];
      |                        ^~~~~~
      |                        lengths
__tester__.c:36:24: note: each undeclared identifier is reported only once for each function it appears in
__tester__.c:42:19: error: ‘lenghts_count’ undeclared (first use in this function); did you mean ‘lengths_count’?
   42 |     for(int i=0;i<lenghts_count;i++){
      |                   ^~~~~~~~~~~~
      |                   lengths_count
```